1 2 9 0

UNIVERSIDADE D
COIMBRA

Relatório realizado para auxílio do trabalho realizado no âmbito da cadeira Sistemas Operativos cujo nome “DEIchain”

Ano letivo 2024/25

Francisco Gouveia – 2023214517 - PL5

João Canais – 2023213000 – PL5

Índice

Funções do sistema 3
- **controller.c** 3
- **statistics.c** 3
- **validator.c** 3
- **miner.c** 3
- **TxGen.c** 4

Resumo e principais funcionalidades 4
- **Controller.c** 4
- **TxGen.c** 4
- **Miner.c** 4
- **Validator.c** 4
- **Statistics.c** 4

O projeto é composto por 6 ficheiros-fonte principais: **controller.c, statistics.c, validator.c, miner.c e TxGen.c** A seguir, apresentam-se as funções centrais de cada ficheiro, apresentado o propósito de cada uma, resumidamente.

Funções do sistema

controller.c

Função	Propósito
`read_conf_file()`	Carrega parâmetros de configuração do ficheiro `config.cfg`
`initialize_resources()`	Cria e inicializa IPC: memória partilhada, semáforos, pipe e a message queue
`initialize_processes()`	Lança processos filhos (validator, miner, statistics) via `fork()`
`realese_resources()`	Liberta todos os recursos IPC e termina processos
`handle_signals()`	Trata do SIGINT e SIGUSR1 no processo controlador
`write_ledger()`	Imprime a blockchain completa na consola e no ficheiro

statistics.c

`read_message_queue()`	Esta a ler message quele à espera de mensagens do validator
`print_statistics()`	Mostra no ecrã as estatísticas
`handle_sigusr1()`	Dispara exibição de estatísticas ao receber SIGUSR1
`main_statistics()`	Loop principal de receção e processamento de estatísticas

validator.c

`verify_Pool()`	Ajusta o número de validadores conforme ocupação da pool
`check_Hash()`	Verifica se `previous_hash` coincide com último bloco do ledger
`transctions_in_Pool()`	Confirma se todas as transações de um bloco ainda estão na pool
`put_block_in_ledger()`	Insere bloco validado no ledger partilhado
`clean_transactions()`	Remove da pool as transações incluídas no bloco
`validate_block()`	Verifica a validação de PoW, hash e transações e aciona a inserção ou não do bloco valido ou inválido
`send_block_to_statistics()`	Envia resultado de validação ao processo de estatísticas
`main_validator()`	Loop principal de receção de blocos pelo pipe e validação contínua

miner.c

`random_difficulty()`	Seleciona aleatoriamente um nível de dificuldade para a mineração
`check_for_transactions()`	Verifica se existem transações disponíveis na pool antes de criar um bloco
`add_transactions_to_block()`	Adiciona transações válidas à estrutura de bloco consoante o nível de dificuldade
`create_block()`	Inicializa campos do bloco, obtém hash anterior e executa Proof-of-Work
`mine(void *arg)`	Função principal de cada thread miner, que gera e envia blocos concluídos para o pipe
`release(int signum)`	Handler de sinal para terminar mineração
`main_miner()`	Inicializa ambiente de threading do miner e aguarda término das threads

TxGen.c

`transaction_maker()`	Cria uma transação e insere-a na pool partilhada
`release()`	Trata o SIGINT, liberta semáforo e desliga memória partilhada
`main()`	Função principal: valida argumentos, liga-se à memória e semáforos

Resumo e principais funcionalidades

Controller.c

O processo Controller começa por ler o ficheiro de configuração com os parâmetros de inicialização e cria em seguida três processos filhos — Miner, Validator e Statistics — bem como as memórias partilhadas, os semáforos, os pipes e a message queue necessários à comunicação entre si. Depois destas inicializações, fica preparado para tratar o sinal SIGINT, que termina todo o programa, e o SIGUSR1, que encaminha um pedido de relatório ao processo Statistics.

TxGen.c

O Txgen é um processo autónomo que recebe dois parâmetros (o valor da reward e um intervalo de tempo) e liga-se à memória partilhada da transaction pool; depois, gera continuamente transações com a reward especificada e intervalos temporais correspondentes ao parâmetro recebido.

Miner.c

O Miner, que é um dos filhos do Controller, lança várias threads — cada uma representando um miner — que escolhem aleatoriamente um valor de reward e procuram transações válidas na pool; caso não consigam reunir dez transações após um certo número de tentativas, diminuem a reward e tentam novamente. Reunidas as dez transações, efectuam o proof of work, gerando hashes até cumprirem o critério de validade, e enviam o bloco resultante através de uma pipe para o Validator.

Validator.c

O Validator lê os blocos da pipe e verifica o proof of work, a existência das transações na transaction pool e a correspondência do previous hash com o último bloco da ledger; se todas as condições estiverem correctas, adiciona o novo bloco à ledger, remove as transações da pool e envia um resumo para a message queue. Sempre que acede à transaction pool, incrementa em 1 a “idade” de cada transação e, ao atingir 50, aumenta também em 1 a sua reward.

Statistics.c

O processo Statistics mantém-se em escuta na message queue para receber as actualizações do Validator, recalcula métricas como o total de blocos processados, número de blocos válidos e inválidos, comprimento da blockchain e a reward acumulada de cada miner (soma das rewards das transações de cada bloco) e, sempre que recebe o sinal SIGUSR1, imprime todas essas informações.

Todos os acessos a memórias partilhadas e a secções críticas estão protegidas por semáforos, garantindo a sincronização e a integridade dos dados.

Tempo de trabalho por aluno – 36h